

Due Date: 08.06.2026, 23:55

CENG 465 Principles of Data-Intensive Systems

Data Replication in a Single-Leader Environment

Objective:

This assignment explores data consistency in a distributed environment through replication techniques. You will implement and test various consistency models within a leader-based replication system and examine the effects of replication lag and concurrent operations on data consistency.

Assignment Steps

1. Environment Setup and Role Assignment

Database Selection: Choose a distributed database system that supports replication, such as PostgreSQL, MongoDB, or Cassandra.

Role Assignment: This is a 2-person group project. Each group must establish a single-leader replication environment:

- **one member must act as the Leader (Primary)**
- **one member must act as the Follower (Secondary)**

Both group members must actively participate in setting up, configuring, and testing the distributed system.

Configuration: Configure replication between the leader and follower, and verify that data changes on the leader are reflected on the follower.

Distributed Environment Requirement:

In this project, students must create a truly distributed environment to simulate and test data consistency in leader-based replication. The system **cannot be installed on a single machine** using local-only configurations (such as Docker with multiple containers or virtualized instances on a single machine). Instead, students must either

- set up the distributed environment across **multiple local machines** within the same network, or
- use a **cloud-based platform** (e.g., Amazon AWS, Google Cloud, Microsoft Azure) to establish and test a multi-node setup.

This requirement is intended to reflect real-world challenges such as replication lag, network latency, and temporary inconsistency in distributed systems.

2. Data Schema and Replication Logging

Define a Schema: Design a schema that is simple in structure but sufficient for replication analysis. The schema must support **insert, update, and delete operations**, and it should include attributes that make data changes observable over time, such as:

- a **version number**
- a **last_updated timestamp**
- an **operation ID** or another update-tracking field

The goal is not only to store data, but also to enable tracking of **update order, visibility, and replication behavior** across the leader and follower.

Replication Logging: Log each write operation with a timestamp and identifier to track data changes and measure replication delays.

Database Operations: Implement code for **insert, update, and delete** operations, with special attention to:

- visibility of updates on the follower
- ordering of data changes across nodes
- tracking how and when updates become visible after they are written on the leader

This part will form the basis of the consistency experiments in the following sections. The original project already expects logging of write operations plus insert/update/delete support for visibility and ordering analysis.

3. Consistency Model Experiments

You will explore three main consistency models: **Eventual Consistency**, **Monotonic Reads**, and **Read-After-Write Consistency**.

Consistency Models to Test:

- **Eventual Consistency:** Over time, all nodes should converge to the same state, though data might be temporarily inconsistent.
- **Monotonic Reads:** Once a client has read a particular state of the data, subsequent reads should not return an older state.
- **Read-After-Write Consistency:** The client should be able to immediately read back their writes.

4. Experiment Steps for Each Consistency Model

Experiment 1: Eventual Consistency

- **Setup:** Write a new record to the leader node and note the write time.

- **Test:** Periodically read the data from the follower node over a fixed period (e.g., every few seconds for one minute) to observe when it converges.
 - **Expected Result:** The follower should eventually show the same data as the leader, although it may lag behind.
 - **Observations:** Document the time it takes for all nodes to reach consistency and analyze any factors contributing to delays.
-

Experiment 2: Monotonic Reads

- **Setup:** Perform a sequence of updates on a single record in the leader node (e.g., increment a "version" field from 1 to 5).
 - **Test:** Sequentially read the record from a follower node.
 - **Expected Result:** Each read should reflect the same or a later version number, preventing "out of order" states.
 - **Observations:** Log any backward reads (e.g., from version 5 to 3) and analyze the cause.
-

Experiment 3: Read-After-Write Consistency

- **Setup:** A client writes a record on the leader node.
 - **Test:** Immediately read the record back from the leader to confirm it reflects the latest write.
 - **Expected Result:** The client should immediately see their write on the leader; other clients may experience a delay on followers.
 - **Observations:** Record the time followers take to reflect the new data.
-

5. Extended Experiments with Consistency Scenarios

Scenario: Concurrent Writes

- **Objective:** Test how concurrent writes to the leader are propagated to followers.
- **Test Steps:**
 - Perform multiple writes in quick succession to the leader.
 - Read from the followers to check if data is seen in the same order.
- **Expected Observations:** Followers should show data in the same sequence as what was written to the leader, but asynchronous replication might cause inconsistencies. Document how different consistency models are impacted.

Progress Presentation – May 5, 2026

By **May 5, 2026**, each group is expected to complete the “**Data Schema and Replication Logging**” part of the project.

During the progress presentation, groups must demonstrate:

- their schema design,
- the fields used to track updates (such as version, timestamp, or operation ID),
- their logging mechanism,
- and sample insert/update/delete operations together with evidence of replication behavior.

Assignment Requirements

1. Write a comprehensive report.
 - The report must include the code **for each step** along with a detailed explanation. The report must emphasize which part of the code corresponds to the requirements of each step. This ensures that the evaluation process can identify how each step's objectives have been met.
2. Provide the code base for your database system in the **appendix** and submit the assignment report according to the stipulated guidelines.
3. **During the presentation**, you must present and demonstrate the tests carried out at each stage of the assignment to show your progress and understanding. Prepare a presentation that follows the outline below (**It should not be more than ten pages**). **Each group member must attend and take part in the presentation.**

Assignment Rules:

- This is a 2-person group assignment. Individual work or more than 2-person group work is not allowed. Also, inter-group collaboration is not allowed!
- Your report must include your names, surnames, and student ID's.
- All assignments are subject to plagiarism detection, and the suspected solutions (derived from or inspired by the solutions of other groups) will be graded as zero.

Submission Rules:

- All submissions must be performed as a zip file via Microsoft Teams by only one of the group members in a zip file.
- Follow a specific naming convention for zip files like **groupNo_CENG465_Project.zip** (Example: **G01_CENG465_Project**) and it should
 - Include a report in a pdf file.
 - Include a presentation in a pdf file.

The content of the report and presentation are mentioned in the “Assignment Requirements” part.